



Parul University

Faculty of Engineering & Technology

BACHELOR OF TECHNOLOGY

Compiler Design (303105350)

3rd Year / 6th Semester

COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

Laboratory Manual

Submitted by:

Name : Sumeet Tiwari
Enroll. No. : 2303051240250
Division : 6B1_AI
Roll No. : 70

Submitted to:

Mrs. Arpita Meet

CERTIFICATE

This is to certify that

~~Mr./Ms.~~ **Sumeet Tiwari** with

*Enrollment no. **2303051240250**..has successfully*

*completed ~~his~~/her laboratory practical in the **Compiler***

Design (303105350) *from the Department of Computer*

Science & Engineering during the academic year

2025 - 26



.....
Date of Submission

.....
Staff In charge

.....
Head of Department

INDEX

Subject Code: 303105350
Subject : Compiler Design

Class: 3rd Year / 6th Sem
Academic Year: 2025 - 2026

S.No.	Practical Title	Page No.		Date of Perf.	Date of Sub.	Marks	Sign
		From	To				
01.	Program to implement Lexical Analyzer.						
02.	Program to count digits, vowels and symbols in C.						
03.	Program to check validation of User Name and Password in C.						
04.	Program to implement Predictive Parsing LL (1) in C.						
05.	Program to implement Recursive Descent Parsing in C.						
06.	Program to implement Operator Precedence Parsing in C.						
07.	Program to implement LALR Parsing in C.						
08.	To Study about Lexical Analyzer Generator (LEX) and Flex (Fast LexicalAnalyzer)						
09.	Implement following programs using Lex. a. Create a Lexer to take input from text file and count no of characters, no. of lines & no. of words. b. Write a Lex program to count number of vowels and consonants in a given input string.						
10.	Implement following programs using Lex. a. Write a Lex program to print out all numbers from the given file. b. Write a Lex program to printout all HTML tags in file. c. Write a Lex program which adds line numbers to the given file and display the same onto the standard output.						

Practical 1

Aim : Program to implement Lexical Analyzer.

Overview : A Lexical Analyzer (Lexer) is the first phase of a compiler.

It scans the source program character by character and groups them into tokens.

Functions of Lexical Analyzer

- Removes whitespace & comments
- Identifies tokens
- Classifies tokens into categories such as:
 - Keyword
 - Identifier
 - Number
 - Operator
 - Special symbol
- Reports lexical errors

Output of Lexical Analyzer

Tokens → passed to the syntax analyzer (parser).

CODE :

```
//Program to implement Lexical Analyzer.
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char str[]) {
    char *keywords[] = {"int", "float", "if", "else", "for", "while", "return"};
    int n = sizeof(keywords) / sizeof(keywords[0]);
    for(int i = 0; i < n; i++) {
        if(strcmp(str, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

int main() {
    char code[100], token[20];
    int i = 0, j = 0;

    printf("Enter code: ");
    fgets(code, sizeof(code), stdin);

    while(code[i] != '\0') {
        if(isalnum(code[i])) {
            token[j++] = code[i];
        }
    }
}
```

```

    } else {
        if(j != 0) {
            token[j] = '\0';
            if(isKeyword(token))
                printf("%s : Keyword\n", token);
            else
                printf("%s : Identifier\n", token);
            j = 0;
        }
        if(code[i] == '+' || code[i] == '-' || code[i] == '*' || code[i] == '/' ||
            code[i] == '=' || code[i] == '(' || code[i] == ')' || code[i] == '{' ||
            code[i] == '}' || code[i] == ';' ) {
            printf("%c : Operator/Symbol\n", code[i]);
        }
    }
    i++;
}
return 0;
}

```

OUTPUT :

```
TERMINAL PORTS
Enter expression: int a = 10;
<KEYWORD , int>
<IDENTIFIER , a>
<OPERATOR , =>
<NUMBER , 10>
<SYMBOL , ;>
```

Practical 2

Aim : Program to count digits, vowels and symbols in C.

Understanding Character Types :

In programming, we often need to analyze text data by categorizing characters into different groups:

- **Digits:** Numeric characters (0-9)
- **Vowels:** Letters that represent vowel sounds in English (a, e, i, o, u and their uppercase versions)
- **Symbols:** All other characters that are not digits, vowels, or consonants (including punctuation, spaces, special characters)

CODE :

```
//Program to count digits, vowels and symbols in C.

#include <stdio.h>
#include <ctype.h>

int main() {
    char str[200];
    int digits = 0, vowels = 0, symbols = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    for (int i = 0; str[i] != '\0'; i++) {
        if (isdigit(str[i])) {
            digits++;
        }
        else if (str[i]=='a' || str[i]=='e' || str[i]=='i' || str[i]=='o' || str[i]=='u' ||
                str[i]=='A' || str[i]=='E' || str[i]=='I' || str[i]=='O' || str[i]=='U') {
            vowels++;
        }
        else if (str[i] != ' ' && str[i] != '\n') {
            symbols++; // any non-space, non-digit, non-vowel char
        }
    }
}
```

```
printf("Digits: %d\n", digits);
printf("Vowels: %d\n", vowels);
printf("Symbols: %d\n", symbols);

return 0;
```

OUTPUT :

```
TERMINAL PORTS
Enter text: My name is manish. This is Practical 2 of CD.

CHARACTER ANALYSIS TABLE

| Category | Count | Characters |
|---|---|---|
| Digits | 1 | 2 |
| Vowels | 11 | a e i a i i i a i a o |
| Symbols | 2 | . . |

Total Characters: 14
```

Practical 3

Aim : Program to check validation of User Name and Password in C.

Username Validation Rules :

- Length between 5-20 characters
- Must start with a letter
- Can contain letters, digits, and underscores only
- No special symbols or spaces

Password Validation Rules:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (@, #, \$, %, etc.)
- No spaces allowed

CODE :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char user[30], pass[30];
    int upper=0, lower=0, digit=0, special=0, valid=1;

    printf("=== USER REGISTRATION ===\n");

    // Username input
    printf("Username: ");
    scanf("%s", user);

    int ulen = strlen(user);

    // Username validation
    if(ulen < 5 || ulen > 20) {
        printf("✗ Username must be 5-20 characters\n");
        valid = 0;
    }
    if(!isalpha(user[0])) {
        printf("✗ Username must start with a letter\n");
        valid = 0;
    }
}
```



```
for(int i=0; i<ulen; i++) {
    if(!isalnum(user[i]) && user[i] != ' _') {

        printf("✗ Username can only contain letters, digits or _\n");
        valid = 0;
        break;
    }
}
// Password input
printf("Password: ");
scanf("%s", pass);

int plen = strlen(pass);

// Password basic validation
if(plen < 8) {
    printf("✗ Password must be at least 8 characters\n");
    valid = 0;
}
// Password character checks
for(int i=0; i<plen; i++) {
    if(isupper(pass[i])) upper = 1;
    else if(islower(pass[i])) lower = 1;
    else if(isdigit(pass[i])) digit = 1;
    else if(pass[i] == ' ') {
        printf("✗ Password cannot contain spaces\n");
        valid = 0;
        break;
    }
    else special = 1;
}
// Summary Table
printf("\n--- VALIDATION SUMMARY ---\n");
printf("Username Valid      : %s\n", (valid && ulen>=5 && ulen<=20 && isalpha(user[0])) ?
"YES" : "NO");
printf("Password Length    : %s\n", plen >= 8 ? "YES" : "NO");
printf("Has Uppercase      : %s\n", upper ? "YES" : "NO");
printf("Has Lowercase     : %s\n", lower ? "YES" : "NO");
printf("Has Digit         : %s\n", digit ? "YES" : "NO");
printf("Has Special Char  : %s\n", special ? "YES" : "NO");
// Final decision
if(valid && upper && lower && digit && special)
    printf("\n✅ Registration Successful!\n");
else
    printf("\n✗ Registration Failed!\n");
return 0;
}
```

OUTPUT :

```
TERMINAL  PORTS  Code + ▾ ⚙️ 🗑️ ⋮ | 🗉 ✕

=== USER REGISTRATION ===
Username: manish
Password: xenon238

--- VALIDATION SUMMARY ---
Username Valid      : YES
Password Length     : YES
Has Uppercase       : NO
Has Lowercase       : YES
Has Digit           : YES
Has Special Char    : NO

✖ Registration Failed!
```

```
TERMINAL  PORTS  Code + ▾ ⚙️ 🗑️ ⋮ | 🗉 ✕

=== USER REGISTRATION ===
Username: manish
Password: Xenon@238

--- VALIDATION SUMMARY ---
Username Valid      : YES
Password Length     : YES
Has Uppercase       : YES
Has Lowercase       : YES
Has Digit           : YES
Has Special Char    : YES

✔ Registration Successful!
```

Practical 4

Aim : Program to implement Predictive Parsing LL(1) in C.

Overview:

LL(1) predictive parsing is a **top-down parsing method** that parses input **left to right**, produces the **leftmost derivation**, and uses **one lookahead symbol** to choose the correct production **without backtracking**.

Steps:

1. Compute **FIRST** and **FOLLOW** sets.
2. Construct the **LL(1) parsing table**.
3. Initialize stack with **Start symbol and \$**.
4. Compare stack top with input symbol and apply table rule.
5. Accept when both stack and input reach **\$**.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 50

char stack[MAX];
int top = -1;

void push(char c) {
    stack[++top] = c;
}

void pop() {
    top--;
}

int isTerminal(char c) {
    return (c == 'i' || c == '+' || c == '$');
}

void displayStack() {
    for (int i = top; i >= 0; i--)
        printf("%c", stack[i]);
}

int main() {
    char input[MAX];
    int ip = 0;

    printf("Enter input string (end with $): ");
    scanf("%s", input);
```

```
push('$');  
push('E');  
printf("\nSTACK\t\tINPUT\t\tACTION\n");
```

```
while (top >= 0) {  
    printf("\n");  
    displayStack();  
    printf("\t\t%s\t\t", &input[ip]);
```

```
    char X = stack[top];  
    char a = input[ip];
```

```
    // If terminal  
    if (isTerminal(X)) {  
        if (X == a) {  
            pop();  
            ip++;  
            printf("Match %c", a);  
        } else {  
            printf("Error");  
            return 0;  
        }  
    }
```

```
    // Non-terminal  
    else {  
        pop();  
        if (X == 'E' && a == 'i') {  
            push('\n');  
            push('T');  
            printf("E -> T E");  
        }  
        else if (X == '\n' && a == '+') {  
            push('\n');  
            push('T');  
            push('+');  
            printf("E' -> + T E");  
        }  
        else if (X == '\n' && a == '$') {  
            printf("E' -> ε");  
        }  
        else if (X == 'T' && a == 'i') {  
            push('i');  
            printf("T -> i");  
        }  
        else {  
            printf("Error");  
            return 0;  
        }  
    }  
}
```

```
printf("\n\nString Accepted!");  
return 0;  
}
```

Output:

Output

Enter input string (end with \$): i+i

STACK	INPUT	ACTION
E\$	i+i	E -> T E'
T'\$	i+i	T -> i
i'\$	i+i	Match i
'\$	+i	E' -> + T E'
+T'\$	+i	Match +
T'\$	i	T -> i
i'\$	i	Match i
'\$		Error

=== Code Execution Successful ===

Practical 5

Aim : Program to implement Recursive Descent Parsing in C.

Overview:

Recursive Descent Parsing is a top-down parsing technique in which each non-terminal of a grammar is implemented as a recursive function.

The parser tries to match the input string starting from the start symbol and moves left to right, producing a leftmost derivation.

How It Works:

- Each function represents a **grammar rule**
- Input is scanned using a **pointer**
- Recursive calls simulate derivation steps
- If input fully matches grammar → **Accepted**

CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char input[50];
int i = 0;

// Function declarations
void E();
void Eprime();
void T();
void Tprime();
void F();

void E() {
    T();
    Eprime();
}

void Eprime() {
    if (input[i] == '+') {
        i++;
        T();
        Eprime();
    }
}

void T() {
    F();
    Tprime();
}
```

```
void Tprime() {
    if (input[i] == '*') {
        i++;
        F();
        Tprime();
    }
}

void F() {
    if (input[i] == '(') {
        i++;
        E();
        if (input[i] == ')')
            i++;
        else {
            printf("Error: Missing )\n");
            exit(0);
        }
    }
    else if (input[i] == 'i' && input[i+1] == 'd') {
        i += 2;
    }
    else {
        printf("Error: Invalid input\n");
        exit(0);
    }
}

int main() {
    printf("Enter the expression: ");
    scanf("%s", input);

    E();

    if (input[i] == '\0')
        printf("String is successfully parsed.\n");
    else
        printf("Error: Invalid string\n");

    return 0;
}
```

Output:

Output

```
Enter the expression: id+id*id
String is successfully parsed.
```


Practical 8

Aim : To study Lexical Analyzer Generator (LEX) and Flex (Fast Lexical Analyzer)

Overview:

1. Download MSYS2 from  <https://www.msys2.org>
2. Install it (default location is fine).
3. After installation, open:
MSYS2 MinGW64 (important, not plain MSYS)s

2.Update MSYS2 Packages (First Time Only)

In **MSYS2 MinGW64 terminal**, run:

pacman -Syu

- If it asks to close terminal → close it
- Reopen MSYS2 MinGW64
- Run again:

pacman -Syu

3.Install FLEX and GCC

Run this once:

pacman -S mingw-w64-x86_64-flex mingw-w64-x86_64-gcc

Check installation:

flex --version

gcc --version

✓ If versions appear → installation successful

4.Create Your Flex Program

Step 1: Create a working folder

mkdir flex_programs

cd flex_programs

Step 2: Create a Lex file

nano simple.l

Paste this simple lexical analyzer example:

```
%{  
#include <stdio.h>  
%}  
%%  
[0-9]+ { printf("NUMBER\n"); }  
[a-zA-Z]+ { printf("IDENTIFIER\n"); }
```

1. Download MSYS2 from  <https://www.msys2.org>
2. Install it (default location is fine).
3. After installation, open:
MSYS2 MinGW64 (important, not plain MSYS)s

2.Update MSYS2 Packages (First Time Only)

In **MSYS2 MinGW64 terminal**, run:

pacman -Syu

- If it asks to close terminal → close it
- Reopen MSYS2 MinGW64
- Run again:

pacman -Syu

3.Install FLEX and GCC

Run this once:

pacman -S mingw-w64-x86_64-flex mingw-w64-x86_64-gcc

Check installation:

flex --version

gcc --version

✓ If versions appear → installation successful

4.Create Your Flex Program

Step 1: Create a working folder

mkdir flex_programs

cd flex_programs

Step 2: Create a Lex file

nano simple.l

Paste this simple lexical analyzer example:

```
%{  
    #include <stdio.h>  
    %}  
%%  
[0-9]+    { printf("NUMBER\n"); }  
[a-zA-Z]+ { printf("IDENTIFIER\n"); }
```

- Press ENTER → file will be saved
- Press Ctrl + X → exit nano

Now you will come back to the MSYS2 terminal prompt like:

~/flex_programs \$

STEP 2: Run FLEX on your file

Type this command:

flex simple.l

✓ If successful, no output will be shown

✓ It will generate a file named:

lex.yy.c

Check (optional):

ls

You should see:

simple.l lex.yy.c

STEP 3: Compile using GCC

Now type:

gcc lex.yy.c -o simple

✓ If no errors → executable created

STEP 4: Run the program

./simple

The program is now waiting for input 

STEP 5: Give input and test

Type:

abc 123 test

Press ENTER, then press Ctrl + D

✓ Output will be:

Identifier

Numbers

Identifier

Output:

```
(kali㉿kali)-[~]
$ cd flex_program
(kali㉿kali)-[~/flex_program]
$ ls
lex.yy.c  simple.l  'simple.l '
(kali㉿kali)-[~/flex_program]
$ nano first.l
(kali㉿kali)-[~/flex_program]
$ flex first.l
(kali㉿kali)-[~/flex_program]
$ gcc lex.yy.c -o first
(kali㉿kali)-[~/flex_program]
$ ./first

abc 123 test
INDENTIFIER
NUMBER
INDENTIFIER

(kali㉿kali)-[~/flex_program]
$
```

```
GNU nano 8.1 se
%{
#include<stdio.h>
int word_count=0;
%}

%%
[a-zA-Z]+ {word_count++;}
/*each word*/
[\t\n]+ { /*ignore spaces*/}
. { /*ignore other characters*/}

%%

int yywrap()
{
    return 1;
}

int main()
{
    yylex();
    printf("Number of words= %d\n", word_count);
    return 0;
}
```

```
(kali㉿kali)-[~/flex_program]
$ nano second.l

(kali㉿kali)-[~/flex_program]
$ flex second.l

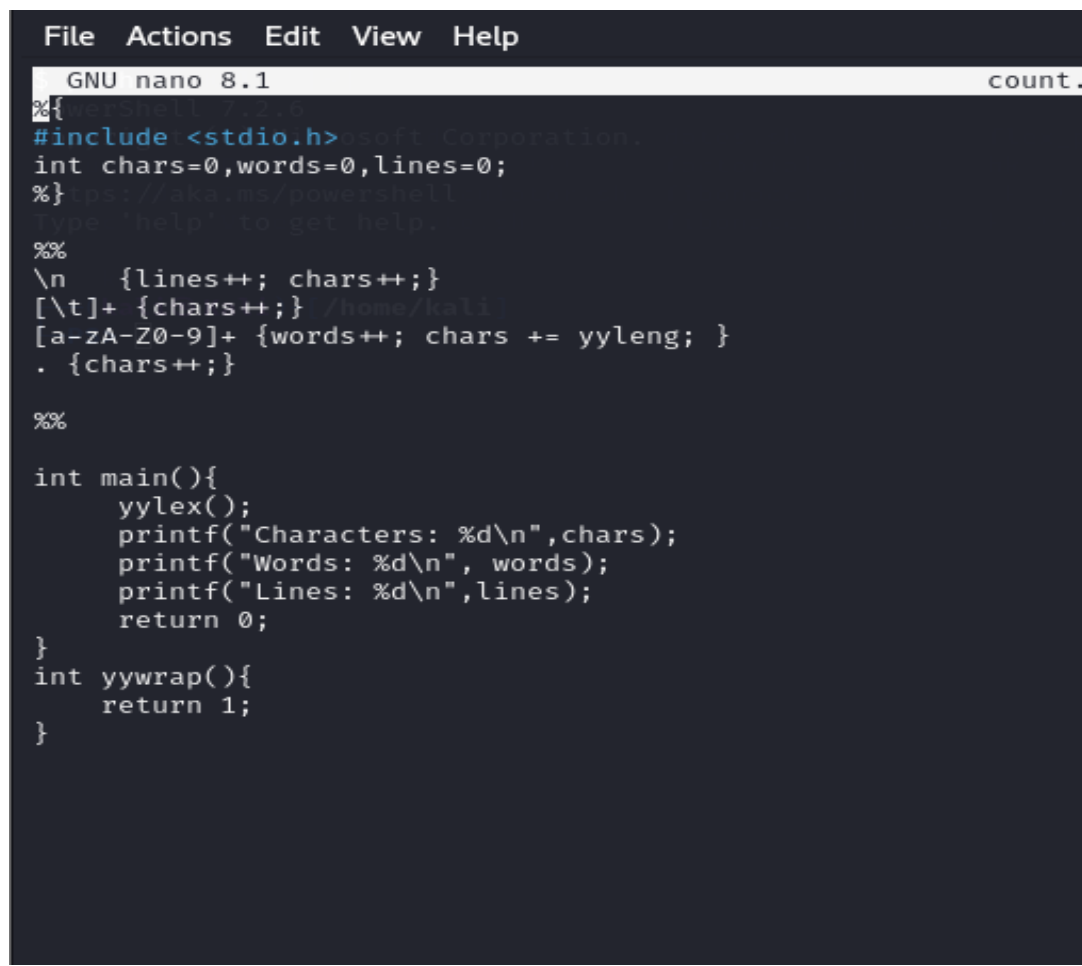
(kali㉿kali)-[~/flex_program]
$ gcc lex.yy.c -o second

(kali㉿kali)-[~/flex_program]
$ ./second
Hello world
this is flex
Number of words= 5
```

Practical 9

Aim: Implement following programs using Lex. a. Create a Lexer to take input from text file and count no of characters, no. of lines & no. of words. b. Write a Lex program to count no. of vowels and consonants.

Program:

A screenshot of a GNU nano 8.1 text editor window. The title bar shows 'GNU nano 8.1' and the filename 'count.l'. The editor contains a Lex program. It starts with a C preprocessor directive '#include <stdio.h>' and initializes three variables: 'chars=0', 'words=0', and 'lines=0'. There are two main rules: one for counting lines and characters, and another for counting words and characters. The first rule is triggered by a newline character '\n' and increments both 'lines' and 'chars'. The second rule is triggered by a sequence of characters '[a-zA-Z0-9]+' and increments 'words' and 'chars' by the length of the match (yyleng). The program ends with a 'main' function that calls 'yylex()' and prints the final counts of characters, words, and lines. There is also a 'yywrap' function that returns 1.

```
File Actions Edit View Help
GNU nano 8.1 count.l
%%{
  #include <stdio.h>
  int chars=0, words=0, lines=0;
}%}
\n {lines++; chars++;}
[\\t]+ {chars++;}
[a-zA-Z0-9]+ {words++; chars += yyleng; }
. {chars++;}

%%

int main(){
    yylex();
    printf("Characters: %d\\n", chars);
    printf("Words: %d\\n", words);
    printf("Lines: %d\\n", lines);
    return 0;
}
int yywrap(){
    return 1;
}
```

OUTPUT:-

```
(kali@Kali)-[~/shell]
$ nano count.l

(kali@Kali)-[~/shell]
$ flex count.l

(kali@Kali)-[~/shell]
$ gcc lex.yy.c -o count

(kali@Kali)-[~/shell]
$ ./count
Hello World

jdk
hjffmfr\
Characters: 26
Words: 4
Lines: 4
```

Program:- Count Vowels and Consonants.

```
GNU nano 8.1 vowel.l
%{
#include<stdio.h>
int vowels=0,consonants=0;
%}
[aeiouAEIOU] {vowels++;}
[a-zA-Z] {consonants++;}
.\n { }
%%

int main(){
    yylex();
    printf("Vowels: %d\n",vowels);
    printf("Consonants: %d\n", consonants);
    return 0;
}

int yywrap(){
    return 1;
}
```

OUTPUT:-

```
(kali㉿Kali)-[~/shell]
$ nano vowel.l

(kali㉿Kali)-[~/shell]
$ flex vowel.l

(kali㉿Kali)-[~/shell]
$ gcc lex.yy.c -o vowel

(kali㉿Kali)-[~/shell]
$ ./vowel
Compiler
Vowels: 3
Consonants: 5
```